

Online Admission System

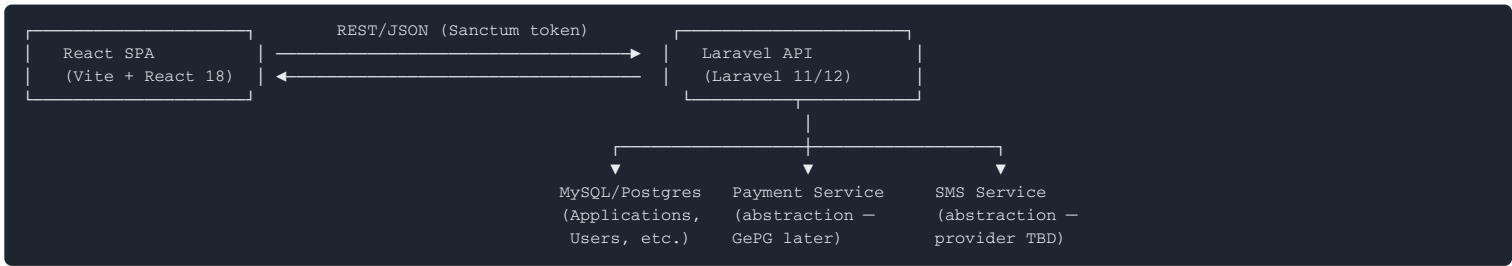
Technical Specification — React + Laravel

Based on Requirements Draft 2.1 & Official Application Form 2026/2027 | September 2026

Stack: React (frontend) + Laravel (backend) · **Design direction:** clean, minimalistic UI **Based on:** Requirements Draft 2.1 + Official Application Form 2026/2027

Assumption stated up front: this spec assumes **Laravel as a REST API backend** (Sanctum for auth) with a **separate React SPA** consuming it — the standard, most flexible pairing for “React + Laravel.” If the team prefers Laravel + Inertia.js instead (React views rendered server-side, one repo, no separate API layer), most of this doc still applies; only the Auth and API sections change. Confirm this before scaffolding.

1. Architecture Overview



Two separate apps, two repos (recommended):

- `mati-uyole-api` — Laravel, exposes REST endpoints only, no Blade views for the applicant/admin flows
- `mati-uyole-web` — React (Vite), consumes the API, deployed separately (or served as static build behind the same domain)

Why separate repos: cleaner deploys, the React team/dev can iterate on UI without touching backend, and it keeps the door open for a future mobile app hitting the same API.

2. Database Schema (Core Entities)

```
users
├── id
├── application_number    -- e.g. MATI-2026-000124 (unique, generated on registration)
├── first_name, middle_name, last_name
├── phone (unique, required)
├── email (nullable)
├── password
├── role                 -- enum: applicant | admission_officer | finance_officer | approving_authority | super_admin
├── email_verified_at
├── created_at, updated_at

applicant_profiles      -- 1:1 with users (role=applicant)
├── id, user_id
├── gender
├── date_of_birth
├── place_of_birth
├── marital_status       -- single | married | other
├── citizenship
├── citizenship_other    -- nullable, when citizenship = "other"
├── region, district, city_town
├── present_address, postal_box
├── has_disability       -- boolean
├── disability_details   -- nullable text
├── passport_photo_path
├── created_at, updated_at

next_of_kin             -- 1:1 with applicant_profiles
├── id, applicant_profile_id
├── full_name, relationship, phone
├── present_address, postal_box, city_town, email

academic_records        -- 1:1 with applicant_profiles
├── id, applicant_profile_id
├── qualification_type   -- CSEE | NVA_III | NTA_DIPLOMA | OTHER
├── index_no_primary     -- format S0000/0000/0000
├── index_no_secondary   -- format P... (confirm meaning with institute)
├── division, points
├── year_completed
├── secondary_school_name, country
├── primary_school_name, primary_school_district, primary_school_region
├── avn_number           -- nullable, NACTVET Award Verification Number
```

```
└─ created_at, updated_at

academic_years
└─ id, label (e.g. "2026/2027"), is_active
└─ application_window_start, application_window_end  -- supports TCU-style rounds; add a 'round' int if multiple windows needed

programmes
└─ id, academic_year_id
└─ name, nta_level, description
└─ capacity, status          -- open | closed
└─ entry_requirements_text    -- or normalize into a separate 'programme_requirements' table if rules need to be queryable

applications
└─ id, user_id, programme_id, academic_year_id
└─ status      -- draft | payment_pending | paid | submitted | under_review
               -- | additional_info_required | eligible | not_eligible
               -- | selected | waitlisted | not_selected | admission_completed
└─ eligibility_result      -- eligible | not_eligible | null (preliminary, system-computed)
└─ submitted_at
└─ created_at, updated_at

payments
└─ id, application_id
└─ control_number          -- shared or per-applicant (see Section 3 decision)
└─ amount, status          -- unpaid | pending | paid | failed | expired
└─ receipt_reference, receipt_file_path
└─ verified_by (user_id, nullable), verified_at
└─ created_at, updated_at

documents                -- post-selection uploads only
└─ id, application_id
└─ type                  -- birth_certificate | academic_certificate | other
└─ file_path, uploaded_at, verified_at

heslb_loan_proofs        -- registration-stage, optional
└─ id, application_id, file_path, submitted_at

sms_logs / notifications
└─ id, user_id, channel (sms|email), template_key, payload, status, sent_at

audit_logs
└─ id, user_id, action, subject_type, subject_id, meta (json), created_at
```

Notes for the dev implementing this:

- Keep `applicant_profiles`, `academic_records`, `next_of_kin` as separate tables (not JSON blobs) — Admission Officers will need to search/filter by fields inside them.
- `applications.status` should be a proper enum (Laravel enum cast or a `statuses` lookup table) — never a free-text string, since reports and dashboards filter heavily on it.
- Add DB-level unique constraint on `(user_id, programme_id, academic_year_id)` in `applications` to prevent duplicate applications to the same programme in the same cycle.

3. Open Decision — Control Number Strategy

Confirm with the institute before building the Payments module:

Option	Effort	Verification
A. Shared control number (current manual process: 997400027916)	Low — no new GePG/NMB setup	Manual — Finance Officer matches uploaded receipt to applicant
B. Per-applicant control number via GePG API	Higher — needs GePG merchant integration	Automatic — webhook/callback marks <code>payments.status = paid</code>

Build the `PaymentService` interface (Section 6) so either option slots in without changing application code.

4. Application State Machine

```
draft → payment_pending → paid → submitted → under_review
→ additional_info_required (loop back to under_review)
→ eligible / not_eligible
→ selected / waitlisted / not_selected
→ admission_completed
```

Enforce transitions server-side (a small state-machine class or Laravel enum + policy), not just in the frontend — e.g. reject any request to submit an application whose payment status isn't `paid`.

5. API Endpoints (v1)

Auth

```
POST /api/v1/auth/register      # phone, password, names → creates user + application_number
POST /api/v1/auth/login
POST /api/v1/auth/logout
POST /api/v1/auth/otp/request   # optional, if OTP-based first login is adopted
```

```
POST    /api/v1/auth/otp/verify
```

Applicant-facing

```
GET      /api/v1/me                                # current applicant + application summary
PUT      /api/v1/me/personal-info
PUT      /api/v1/me/next-of-kin
PUT      /api/v1/me/academic-info
GET      /api/v1/programmes                    # list open programmes for active academic year
POST     /api/v1/applications                  # create application (selects programme)
GET      /api/v1/applications/{id}
POST     /api/v1/applications/{id}/eligibility-check
POST     /api/v1/applications/{id}/submit
GET      /api/v1/applications/{id}/status
POST     /api/v1/payments/{application_id}/initiate
GET      /api/v1/payments/{application_id}
POST     /api/v1/documents                    # post-selection uploads only
```

Admin/Institute-facing (guarded by role middleware)

```
GET      /api/v1/admin/applications            # filterable: status, programme, region, district, gender, payment_status
GET      /api/v1/admin/applications/{id}
PATCH   /api/v1/admin/applications/{id}/status
POST     /api/v1/admin/applications/{id}/request-info
GET      /api/v1/admin/payments                # filter unpaid/pending/paid
PATCH   /api/v1/admin/payments/{id}/verify    # manual verification path (Option A above)
POST     /api/v1/admin/programmes
PATCH   /api/v1/admin/programmes/{id}
POST     /api/v1/admin/selection/run           # applies selection criteria to eligible pool
GET      /api/v1/admin/reports/applications
GET      /api/v1/admin/reports/payments        # ?format=csv|pdf
```

Use **Laravel Sanctum** for token auth (SPA-friendly), **Form Requests** for validation (see Section 7), and **API Resources** for consistent JSON shape.

6. Integration Abstraction Interfaces

Define these as interfaces/contracts from day one, even with a stub/manual implementation underneath — this is what makes future API integration a swap, not a rewrite.

```
interface PaymentServiceContract {
    public function generateControlNumber(Application $application): string;
    public function checkStatus(string $controlNumber): PaymentStatus;
    // Option A (manual): checkStatus reads from a manually-updated 'payments' row
    // Option B (GePG): checkStatus calls the GePG API / handles their webhook
}

interface SmsServiceContract {
    public function send(string $phone, string $templateKey, array $data): void;
}

interface AcademicVerificationContract {
    public function verifyIndexNumber(string $indexNo): VerificationResult;
    // Stub returns "unverified — manual" until NECTA/NACTVET API access exists
}
```

Bind these in a Laravel service provider so swapping the underlying implementation is a one-line config change.

7. Validation Rules (Form Requests)

```
'index_no_primary' => ['required', 'regex:/^S\d{4}\d{4}\d{4}$/'],
'index_no_secondary' => ['nullable', 'regex:/^P\d{4}\d{4}\d{4}$/'], // confirm exact format with institute
'phone' => ['required', 'regex:/^0\d{9}$|^255\d{9}$/'],
'avn_number' => ['nullable', 'regex:/^d{2}[A-Z]{2}\d+[A-Z]{2}$/'], // e.g. 19NA12212ME — confirm exact pattern with NACTVET
'passport_photo' => ['required', 'image', 'max:2048'], // application-stage upload
'documents.*.file' => ['required', 'mimes:pdf,jpg,png', 'max:5120'], // post-selection only
```

8. UI / Design Direction — Clean & Minimalistic

Since the brief calls for **very clean, minimalistic** design, keep these principles in front of whoever builds the React screens:

- **One accent color, used sparingly** — e.g. a single green (echoing agriculture/MATI branding) for primary actions and status badges. Everything else stays neutral (white/off-white background, dark grey text, light grey borders).
- **Generous white space over dense forms.** The multi-step flow (Account → Personal → Academic → Payment → Programme → Review) should be **one step per screen**, not one long scrolling form — this matches “student asijazwe form ndefu” from the requirements doc directly.
- **Progress indicator** (simple stepper, not a percentage bar) so the applicant always knows where they are and what’s left.
- **Typography:** one clean sans-serif family (e.g. Inter or system font stack), 2–3 weights max, no decorative fonts.
- **Status badges** (Submitted / Under Review / Selected / etc.) as small, color-coded pills — this is the one place color-coding earns its keep, since applicants will scan for it.
- **No unnecessary icons or illustrations** on data-entry screens — save any illustration/branding moment for the dashboard’s empty/welcome state only.
- **Mobile-first:** most applicants will use phones. Design the single-column mobile layout first, then adapt up to desktop — not the reverse.

9. Roles & Permissions Matrix

Action	Applicant	Admission Officer	Finance Officer	Approving Authority	Super Admin
View/edit own application	✓	-	-	-	-
View all applications	-	✓	✓ (payment view only)	✓	✓
Change application status	-	✓	-	-	✓
Verify payments	-	-	✓	-	✓
Run/approve selection	-	-	-	✓	✓
Manage programmes/requirements	-	-	-	-	✓
Manage users/roles	-	-	-	-	✓
View reports	-	✓	✓	✓	✓

Implement with Laravel policies/gates keyed off `users.role` , not scattered `if` checks in controllers.

10. Suggested Build Order (MVP)

1. Auth + user registration + application_number generation
2. Applicant profile, next-of-kin, academic-record forms (React step flow)
3. Programmes CRUD (admin) + programme listing (applicant)
4. Payment module with `PaymentServiceContract` stub (Option A manual path first)
5. Application create/submit + state machine + eligibility check (rule-based, per Section D of the form)
6. Admin application list/filter/review + status changes
7. SMS notification triggers on key state changes
8. Reports (applications, payments) — CSV first, PDF later
9. Post-selection: document upload, admission letter generation
10. Audit log (wrap around steps 5-6 as they’re built, not bolted on later)

11. Non-Functional Notes

- **Security:** rate-limit auth endpoints, hash passwords (Laravel default bcrypt), validate all uploads server-side (mime + size), never trust client-side eligibility checks alone — recompute server-side before allowing submit.
- **Audit everything that changes application state** — who, when, from what status to what status.
- **Swahili + English:** since applicant-facing copy is in Swahili, keep all user-facing strings in a translation file (`resources/lang` or a JSON i18n file on the React side) from day one, not hardcoded — makes future English toggle or wording fixes trivial.
- **Environments:** confirm hosting target early (local XAMPP dev is fine, but production will need a real MySQL/Postgres host + HTTPS for GePG compliance if Option B is chosen later).